演習04：鍵・扉・スイッチ

担当案：メンバーB / 氏名は話し合いで記入

到達目標

鍵の色に対応する扉を開き、スイッチで別の通路を開ける仕組みを作ります。取得・解錠は一度だけ起こり、表示と衝突判定が一致する状態を目指します。

既存の鍵3個の自動取得は共通ベースです。新規部分は鍵IDによる解錠、扉の通行状態、Fキーのスイッチ操作です。先に00_commonの4〜9節とWorld APIを読みます。

必須：色付き鍵、対応する扉、スイッチ、二重処理防止。任意：到達可能性を検査した鍵のランダム配置。

練習1：データと状態を分ける

Stage.keysは {鍵ID: Cell}、DoorSpecはdoor_id・cell・key_id、SwitchSpecはswitch_id・cell・door_idsを持ちます。key_id=Noneの扉はスイッチ専用です。

Gimmicks(world)を作り、worldへの参照、inventory=set()、used_switches=set()を初期化します。扉の開放済み状態はWorldだけで管理し、Gimmicksに別の開閉フラグを作りません。

```
DoorSpec('red_gate', (13, 5), key_id='red')
SwitchSpec('switch_a', (12, 3), door_ids=('side_gate',))
```

座標は説明例です。実際のStageで進行可能になる配置を代表者と決め、validate_stageで検査してください。鍵はその鍵で開く扉の外側へ置きます。

練習2：鍵を取得する

update(player)で、hiddenなら何もせず戻ります。stage.keysを走査し、未取得かつ24px以内の鍵IDだけをinventoryへ追加します。

```
for key_id, cell in self.world.stage.keys.items():
    if key_id in self.inventory:
        continue
    distance = player.position.distance_to(self.world.center(cell))
    # TODO: 距離を確認して取得する
```

取得した鍵を消す際、元のStage.keysから削除しないでください。Stageは配置データです。描画時にinventoryへ入っている鍵を省けば、再挑戦で同じStageを再利用できます。

Game側は更新前後のinventoryの差分を見てMissionTracker.on_keyへ通知します。Gimmicksからスコアを直接増やさないでください。

練習3：鍵付きの扉を開ける

主人公が扉の中心から40px以内で、対応するkey_idをinventoryに持っていればworld.open_doorを呼びます。扉の中に入る必要はありません。壁として塞がれていても隣から開けられる距離です。

key_id=Noneの扉はこの処理では開けません。鍵は開けても消費しない仕様にします。出口でも必須鍵を確認できるためです。

初めて開いたときだけメッセージや効果を出します。open_doorの返り値がFalseなら何もしません。

確認：鍵なしでは通れず、対応する鍵を拾って近づくと通れる。別の色の鍵では開かない。

練習4：Fキーでスイッチを押す

handle_event(event, player)でKEYDOWNかつK_fを判定します。hidden中は操作しません。主人公から24px以内にある未使用スイッチを最も近いもの1つ選びます。

対象のdoor_idsを順にworld.open_doorへ渡し、switch_idをused_switchesへ追加します。今回の必須課題では開放のみで、もう一度押しても閉じません。

Eは隠れる操作で使用します。スイッチにはFを使い、操作案内にも表示してください。

確認：遠いスイッチは反応せず、近いスイッチを押すと指定した扉だけが開く。

練習5：出口と描画を接続する

can_exitは必要な鍵が全てそろっているかだけを返します。位置の判定はGame.check_resultが行います。

```
def can_exit(self) -> bool:
    return self.world.stage.required_keys <= self.inventory
```

旧Game.keysの参照を検索し、collect_items、check_result、draw_map、draw_hud、結果表示をinventoryへ置き換えます。固定の3ではなくrequired_keysやStage.keysの長さを使います。

扉の見た目はWorld側、鍵とスイッチはGimmicks.drawで描く分担です。inventoryにある鍵は描かず、使用済みスイッチは色を変えます。

Game.collect_itemsにある回復処理は消さず、担当範囲の境界を代表者に伝えます。

練習6：進行不能を防ぐ

代表者のvalidate_stageへ仕掛けデータを渡します。鍵を使うたびに行ける範囲が広がることを確認してください。

最初は「赤い鍵で廊下の扉を開く」「奥のスイッチで別の扉を開く」の2段階にします。全仕掛けを一度に配置すると、どの扉が原因か分かりにくくなります。

ランダム配置は任意課題です。採用するなら、生成後に進行可能性を検査し、試行回数に上限を置きます。無限に再抽選しないでください。

動作確認表

準備・操作	期待する結果
同じ鍵に2回触れる	inventoryとミッション通知は1回分
青い鍵で赤い扉へ	扉は開かない
赤い鍵で赤い扉の隣へ	壁と見た目が同時に変わる
開いた扉へもう一度近づく	二重の加点や通知がない
hidden中に鍵・スイッチへ	取得・操作しない
スイッチを2回押す	初回だけ開放、扉は閉じない
必須鍵の一部だけで出口へ	クリアしない
再挑戦	鍵、スイッチ、扉が初期状態

よくある不具合と提出

扉の絵だけ消えて通れない場合はWorld.wallsの更新を確認します。主人公は通れるのに敵が通れない場合はfind_pathが古いGRIDを参照していないか確認します。

再挑戦しても鍵が戻らない場合はStage.keysを直接削除していないかを見ます。

gimmicks.py、配置変更、READMEをコミットします。PRには鍵IDとdoor_idの対応、Fキー、Worldへの依存、動作確認結果を記載します。

基準コード：https://github.com/kusupon/ProjExD_Group13/blob/33fe9905c183b321d1988d66041feb8b7e74998f/game.py